

User manual

1.Introduction

The User Manual contains all essential information for the user to make full use of the information system. This manual includes a description of the system functions and capabilities, contingencies and alternate modes of operation, and step-by-step procedures for system access and use. The manual format may be altered if another format is more suitable for the particular project.

1.1 Purpose

The purpose of this user manual is aimed at helping users to have a more convenient operation on our system. And also guide users to explore the capability for the system.

1.2 Organization

The organization of this user manual is roughly consist of 5 part. The first part is for introduction. The second part is for the details of system capabilities. The third part is the description of system function. The forth part is the details for operating instructions. The fifth part roughly explain the error handling.

2. System capability

This section will provide a brief overview of the system and its capability

2.1 Purpose

The purpose of designing this command-line-based interpreter system is mainly to help users create some simple programs with some easy-understanding language. The system also aimed at offering easy-operating

interpreter system to raise the interest for beginners in programming. In addition, it is also design for young students who are interest in program, and help their future development.

2.2 General description

The brief overview of the this system will be presented as follows

1. Expression definition

The system support user to define a certain kind of expression

This includes:

1. binary expression

binary expression `binexpr` create a binary operation between two expression

i.e., `+` `-` `*` `%` is the operation used between two int expression and return int result

`==` `!=` is the operation used between both int and bool return bool result

`>=` `>` `<` `<=` is the operation used between two int and return bool result

`||` `&&` is the operation used between bool and return bool result

```
>vardef vardef1 int x 1
>binexpr exp1 x + 1
>binexpr exp2 x / 2
>binexpr exp3 x * 3
>
```

2. unary expression

unary expression `unexpr` create a unary operation to operate a certain expression

`#` `~` is the operation used for int and return int result

`!` is the operation used for bool and return bool result

```

>vardef vardef1 int x 3
>vardef vardef2 bool y false
>unexpr unexpr1 # x
>unexpr unexpr2 ~ x
>unexpr unexpr3 ! y
>|

```

3. variable definition

vardef defines a bool or int variable with value from an expression

```

>vardef vardef1 int x 0
>vardef vardef1 bool y true
>

```

2. Statement definition

The system allows users to define a series of different statements

1. Assign statement

Assign a certain constant value, binary expression value, unary expression value or a variable value to another variable

```

>vardef vardef1 int x 3
>vardef vardef2 bool y false
>unexpr unexpr1 # x
>unexpr unexpr2 ~ x
>unexpr unexpr3 ! y
>assign assign1 x unexpr1
>assign assign2 y unexpr3
>

```

2. Print statement

Print the result of an expression

```
>print print1 unexpr1  
>print print2 y
```

3. Skip statement

Does nothing

```
>skip skip1
```

4. Block statement

Merge a list of statement to consist a block

```
>block block1 assign1 assign2 print1
```

5. If statement

This operation judge the bool value of an expression and decide which statement to do next

```
>binexpr exp1 x < 3  
>if if1 exp1 block1 print1
```

6. While statement

The statement judge whether the statement inside the while body can execute depends on the bool value of an expression

```
>while while1 exp1 block1
```

7. Program statement

This statement defines a new Simple program with a unique name , and the program has the statement as its body

```
>block block2 if1 while1  
>program program1 block2
```

3. Program operation

1. execute

this operation execute the program

```
>execute program1
```

2. List

List all the command form the program

```
>list program1
```

3. Store

Store the program as txt file to a certain path

```
>store program1 d://
```

4. Load

Read a txt file contains command from a path and load it to the system as a new program

```
>load program2 d://program1
```

5. Quit

Quit the program

```
>quit|
```

```
Process finished with exit code 0
```

3.The details of operation (running Code sample)

```
vardef vardef1 int x 7
```

define a parameter int x = 7

```
vardef vardef1 int x 7  
vardef vardef2 int y 0
```

Define a parameter int y = 0

```
vardef vardef1 int x 7  
vardef vardef2 int y 0  
binexpr exp1 x * 3
```

Set the expression exp1 = x * 3

We should notice that the binexpr
represent the binary operation, that is like X + Y;

```
vardef vardef1 int x 7  
vardef vardef2 int y 0  
binexpr exp1 x * 3  
binexpr exp2 y + 1
```

Set another expression exp2 = y + 3

```
vardef vardef1 int x 7  
vardef vardef2 int y 0  
binexpr exp1 x * 3  
binexpr exp2 y + 1  
binexpr exp3 y <= 0
```

Set $\text{exp3} = (y \leq 6)$, return the boolean value

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
```

Print the value of x

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
```

Equal to $\text{assign1} = (x = x * 3)$, assign1 represent the container for storage

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
assign assign2 y exp2
```

The same as the last operation

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
assign assign2 y exp2
block block1 print1 assign1 assign2
```

Use block to Collect commands in a block1

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
assign assign2 y exp2
block block1 print1 assign1 assign2
while while1 exp3 block1
```

Use while to start loop,just like:

While(exp3){block1}

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
assign assign2 y exp2
block block1 print1 assign1 assign2
while while1 exp3 block1
block block2 vardef1 vardef2 while1
```

Use the block to contain vardef1 vardef2 while1 in the block2

```
vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
assign assign2 y exp2
block block1 print1 assign1 assign2
while while1 exp3 block1
block block2 vardef1 vardef2 while1
program program1 block2
```

Use the program to contain the block(s) in it


```

vardef vardef1 int x 7
vardef vardef2 int y 0
binexpr exp1 x * 3
binexpr exp2 y + 1
binexpr exp3 y <= 6
print print1 x
assign assign1 x exp1
assign assign2 y exp2
block block1 print1 assign1 assign2
while while1 exp3 block1
block block2 vardef1 vardef2 while1
program program1 block2
execute program1

```

Execute all the commands in the program1

4. Operation instructions

Initiate Operation

The user first enters a blank cmd interface, at this time the user needs to enter and exit different commands to enter different coding modules. Users generally input Vardef operations, and other operations may result in no object operations and errors due to undefined variables.

Maintain Operation

When an error occurs in the operation, the general program will recycle and require the user to re-input until the user's input is judged to be correct.

Terminate and Restart Operations

When the error is not in the interaction with the user (that is, user input), but in the error at runtime, the program will grab the wrong track and print it out, and the program will end abnormally. And the user needs to run the system again if he needs to use it again.

5. ERROR HANDLING

1.Vardef

When the wrong type is entered, it will be automatically recognized and "wrong type" will be returned

2.Binexpr

When the operation symbol is not the same as the symbol contained in the program, or the dividend is zero (eg. $x / 0$ or $x \% 0$), the program reports an error.

3.Unexpr

The error occurs when the input operation is not unary operation or the expression reference do not accept that kind of operating. The system will return error report and ask user to input again.

4.Assign

The error occurs when users' input do not meet the requirement. Like they input variable that not exist, expression not exist or the command format mistake. The system returns corresponding report and ask user to input again.

5.Print

Error occurs when expression reference dose not exist or wrong command format. The system returns corresponding report and ask user to input again.

6.Skip

Errors occur when wrong command format. The system returns corresponding report and ask user to input again.

7.Block

Errors occur when wrong command format. The system returns corresponding report and ask user to input again.

8.If

Error occurs when there is no meetable expression reference or labstatement or wrong command format. The system returns corresponding to report and ask user to input again.

9.While

Error occurs when the statement lable or expression reference dose not exit. Also occurs when the type of value of expression reference is not bool. The system returns corresponding report and ask user to input again.

10.Program

Errors occur when the statementLab dose not exist or command format wrong

Errors occur when wrong command format. The system returns corresponding report and ask user to input again.

11.Execute

Errors occur when ProgamName can not be found or command format wrong. The system returns corresponding report and ask user to input again.

12.List

When programname is not found, report back and ask the user to enter it again. When there is an error in the file operation, use try-catch to catch the error and return StackTrace.

13.Store

When programname is not found, report back and ask the user to enter it again. When there is an error in the file operation, use try-catch to catch the error and return StackTrace.

14.Load

Errors occur when the name“quit”is not corresponding, The system returns corresponding report and ask user to input again.